DECOY WALLET APPLICATION

with Intruder Detection and Secure Alert System

An Android security application that disguises itself as a normal UPI wallet and silently captures details of anyone who tries to break in.

Project Title	Decoy Wallet Application - Intruder Detection and Secure Alert System
Platform	Android (Kotlin)
Document	Project Documentation / Technical Report
Version	1.0
Date	October 2026

Submitted by : _____

Roll No. / ID : _____

Guide / Faculty : _____

Department : _____

Table of Contents

1. Introduction 4

2. Problem Statement and Motivation 4

3. Objectives of the Project 4

4. Scope of the Project 5

5. Existing Systems and Why a Decoy Approach 5

6. Proposed System Overview 6

7. How the Application Works 6

7.1 The disguise: normal everyday use . . . 6

7.2 Wrong PIN: the intruder detection flow . . . 6

7.3 Owner access flow . . . 7

8. Technology Stack 7

9. System Architecture 8

10. Module and Screen Descriptions 9

10.1 LoginActivity - the decoy lock screen . . . 9

10.2 DecoyWalletActivity - the fake wallet . . . 9

10.3 Payment screens - Send, Add Money, Scan, Bills, Pay . . . 9

10.4 OwnerDashboardActivity - the Security Center . . . 9

10.5 IntruderDetailActivity - one alert in full . . . 10

10.6 SettingsActivity - controlling the protection . . . 10

11. Data Model and State Management 10

12. User Interface Design 11

13. Security Design and Considerations 11

14. Development and Build Process 12

15. Testing 12

16. Project Folder Structure 13

17. Limitations 13

18. Future Enhancements 14

19. Ethical and Legal Considerations **14**

20. Conclusion **14**

21. References **15**

1. Introduction

Almost everyone today keeps their whole life on their phone. Banking apps, UPI, photos, chats, work mail, saved passwords, it is all sitting behind a four digit PIN. That makes a stolen or snatched phone a much bigger problem than the cost of the handset itself. The real worry is what the thief can open once the device is in their hands.

This project started from a simple question: instead of only trying to keep an intruder out, what if the phone could quietly turn the tables and gather evidence about the person trying to get in? The Decoy Wallet Application is built around that idea. On the surface it looks and behaves like an ordinary UPI wallet, the kind of app nobody looks at twice. But the moment a wrong PIN is entered, the app treats that person as an intruder: it silently records who they are and where they are, alerts the real owner, and then shows the intruder a believable fake wallet so they never realise they have been caught.

This document describes the project in full, the thinking behind it, the technology used, how each screen works, how the app is put together internally, how it was built and tested, and what is left for future work. The Android application described here is the front end of the system. The sensitive hardware parts (actually firing the camera, reading GPS, and sending alerts over the internet) are described as part of the design and are marked clearly wherever they are planned rather than already wired in.

2. Problem Statement and Motivation

Phone theft is common and the financial loss attached to it has grown a lot because of UPI and mobile banking. A normal lock screen only does one thing: it blocks access. If someone guesses or shoulder surfs the PIN, that single wall is gone and there is nothing underneath it. The owner also has no idea who took the phone, because the device gives nothing away during a failed unlock attempt.

Standard anti theft features exist (remote lock, remote wipe, location tracking) but they usually need the owner to first realise the phone is gone, log in somewhere else, and react in time. By then the intruder may already be inside. There is a gap here. Nothing on a typical phone actively collects information about the intruder at the exact moment of the break in attempt, and nothing deceives the intruder into thinking they succeeded.

The motivation for this project is to fill that gap with a proactive, deception based layer of security. Rather than just failing silently, the app uses the failed attempt as a trigger to capture evidence and to mislead. This gives the genuine owner something concrete to act on (a photo, a location, a time) and buys time by keeping the intruder busy with a decoy.

3. Objectives of the Project

The main objectives we set for the project are:

1. To build an Android application that convincingly disguises itself as a normal UPI/wallet app so that an intruder does not suspect it is a security tool.
2. To detect an unauthorised access attempt automatically whenever a wrong PIN is entered.
3. To capture evidence about the intruder at that moment, specifically a front camera photo, the current GPS location, the date and time, and basic device information.

4. To alert the genuine owner through channels such as email, SMS and WhatsApp, and to keep a record of every attempt.
5. To deceive the intruder after a wrong PIN by showing a fully working but fake wallet, so the intruder believes they are in and does not try anything more aggressive.
6. To give the real owner a private Security Center where all intruder alerts can be reviewed, along with settings to control how the protection behaves.
7. To design a clean, modern and trustworthy looking interface, since a wallet app that looks fake would defeat the whole purpose.

4. Scope of the Project

The project is an Android application targeting ordinary smartphones. It covers the complete user facing side of the system: the decoy login, the fake wallet with working Send, Add Money, Scan and Bill payment flows, the owner Security Center with the list of intruder alerts and their details, and a settings screen for controlling the protection features.

What the current build focuses on is the application logic and the user experience. The parts that touch sensitive hardware and the network, namely the silent camera capture, the GPS read, the encrypted upload to a server and the actual sending of email, SMS and WhatsApp alerts, are part of the overall design and are explained in this document, but in this build they are represented with clearly marked placeholders and realistic sample data. This keeps the project easy to demonstrate and review while still showing exactly where and how the real services would plug in.

5. Existing Systems and Why a Decoy Approach

There are a few things already available that overlap with parts of this idea, so it is worth being honest about where they stop.

Built in anti theft (Find My Device, Find My iPhone)

These let an owner locate, ring, lock or wipe a lost phone from another device. They are useful, but they are reactive. The owner has to notice the loss and act. They also do not identify the person holding the phone and they do not try to deceive that person.

Intruder selfie features

Some custom Android versions and third party lockers can take a photo after a few failed unlock attempts. This is close in spirit to one part of our project, but it is usually just the photo. It is not combined with location, a deceptive decoy, owner alerts over multiple channels, and a disguised front end.

App lockers and vaults

Vault apps hide photos or files behind a second password. The weakness is that they often look like what they are (a lock icon, a vault name), which tells an intruder exactly where the valuable things are kept.

The decoy approach is what makes this project different. By combining disguise, evidence capture, owner alerting and an intentionally convincing fake wallet into a single app, the system does not just

resist an intruder, it studies them and wastes their time while quietly informing the owner.

6. Proposed System Overview

The proposed system is a single Android app with two faces. To the world, and to any intruder, it is **SecureWallet**, a normal looking UPI wallet with a PIN lock. Internally it also contains a hidden owner side and an intruder detection mechanism that the attacker never sees.

The behaviour depends entirely on the PIN that is entered on the lock screen:

- **Correct owner PIN** opens the real side of the app, the Security Center, where the owner can review every intruder attempt and change settings.
- **Any wrong PIN** is treated as an intruder. The app records the attempt, (in the full system) captures a photo and location and sends alerts, and then opens a convincing fake wallet so the intruder is kept busy and unaware.

Because both behaviours live inside one innocent looking app, the security function stays hidden. An intruder who enters a guessed PIN is simply shown a wallet with a healthy balance and a normal looking transaction history, which is exactly what they expect to see, so they have no reason to dig further.

7. How the Application Works

This section walks through the three main flows in plain language. The screen names in brackets map directly to the Activity classes in the code, which are described in detail later.

7.1 The disguise: normal everyday use

When the app is opened it shows the SecureWallet lock screen (LoginActivity): a shield logo, the app name, four PIN dots and a numeric keypad with a fingerprint key and a backspace key. There is nothing on this screen that hints at a security or anti theft tool. It looks like the login of any banking app. The genuine owner simply types their PIN and is taken straight to whichever side matches that PIN.

7.2 Wrong PIN: the intruder detection flow

The interesting part happens when the PIN is wrong. From the intruder's point of view they just entered a PIN and the wallet opened, nothing unusual. Behind that, in the moment between the wrong PIN and the wallet appearing, the system does several things in sequence:

1. The failed attempt is logged as a new intruder record (handled by IntruderStore in the app).
2. The front camera is triggered silently to capture a photo of whoever is holding the phone, with no shutter sound, flash or preview. (Planned; marked as a placeholder in the current build.)
3. The current GPS location, the exact date and time, and the device model and Android version are collected and attached to that record. (Planned.)
4. This evidence is encrypted and sent to the owner over the configured channels (email, SMS, WhatsApp) and optionally uploaded to a secure cloud server for safe keeping. (Planned.)
5. Finally the intruder is dropped into the decoy wallet (DecoyWalletActivity), a fully working but fake wallet, so nothing looks suspicious.

The decoy wallet is deliberately detailed. It shows a realistic balance, a UPI ID, a masked card number and a list of believable past transactions. Its Send, Add Money, Scan and Bills buttons all work and update the on screen balance and history, so if the intruder pokes around, everything responds like a real app. None of it touches real money or a real bank; it is all self contained fake data. To stop the intruder from wandering back into the login or owner side, the device back button from the decoy wallet simply sends the app to the background instead of returning to the lock screen.

7.3 Owner access flow

When the correct owner PIN is entered, the app opens the Security Center (OwnerDashboardActivity) instead of the wallet. This is the private control room that the intruder never reaches. It shows whether protection is active, a count of intruder attempts and alerts sent, and a scrollable list of every intruder alert. Tapping any alert opens its full details (IntruderDetailActivity): the captured snapshot, the location with coordinates, the timestamp, the device, how many times they tried, and which alert channels were used. From the Security Center the owner can also open Settings (SettingsActivity) to turn individual protections on or off, pick which decoy interface to show, and choose the alert channels.

8. Technology Stack

The application is a native Android project written in Kotlin. The choices below were made to keep the app modern, lightweight and easy to run on a normal student machine or college lab, without pulling in heavy external dependencies.

Layer	Technology	Why it was chosen
Language	Kotlin 1.9.24	Official, modern language for Android; concise and safe.
UI toolkit	Android Views + XML layouts	Stable, well documented, easy to lay out screens.
Design system	Material Components (Material 3), v1.12.0	Gives the clean, modern look expected of a banking app.
View access	ViewBinding	Type safe access to views without findViewById boilerplate.
Min / Target SDK	minSdk 24 (Android 7.0), targetSdk 34 (Android 14)	Runs on the vast majority of in-use devices.
Build system	Gradle 8.7 with Android Gradle Plugin 8.5.2	Standard Android build tooling.
JDK	Java 17	Required by the current Android Gradle Plugin.
IDE	Android Studio	Standard environment for Android development.
Testing device	Android Emulator (Pixel 6a, API 35)	Used to run and verify the app during development.

Table 8.1 - Technology stack used in the project.

A small number of AndroidX libraries are used on top of the above: AppCompat, ConstraintLayout, RecyclerView, CardView and GridLayout. These are standard support libraries that almost every Android app relies on.

Planned back-end technologies

For the parts that are designed but not wired in yet, the intended technologies are: the Android CameraX or Camera2 API for the silent capture, the Fused Location Provider for GPS, an email/SMS/WhatsApp gateway (for example an SMTP service, an SMS API, and the WhatsApp Business API) for alerts, and a secure cloud backend such as Firebase for storing the encrypted evidence. Data in transit would use HTTPS/TLS and the stored evidence would be encrypted before upload.

9. System Architecture

The app follows a straightforward, layered structure. Each screen is an Activity with its own XML layout. Shared state (the wallet balance, the transaction history and the list of intruder alerts) is kept in small in memory store objects that any screen can read from and write to. This keeps the code simple to follow, which matters for a project that has to be explained and defended.

At a high level the system is made of four parts:

- **Presentation layer** - the Activities and XML layouts that the user sees and interacts with.
- **State / data layer** - the WalletStore and IntruderStore objects that hold the current data and the logic that changes it.
- **Security / detection logic** - the decision made on the login screen (owner vs intruder) and the evidence capture that follows a wrong PIN.
- **External services (planned)** - camera, location, alert gateways and the cloud backend that the security logic would call out to.

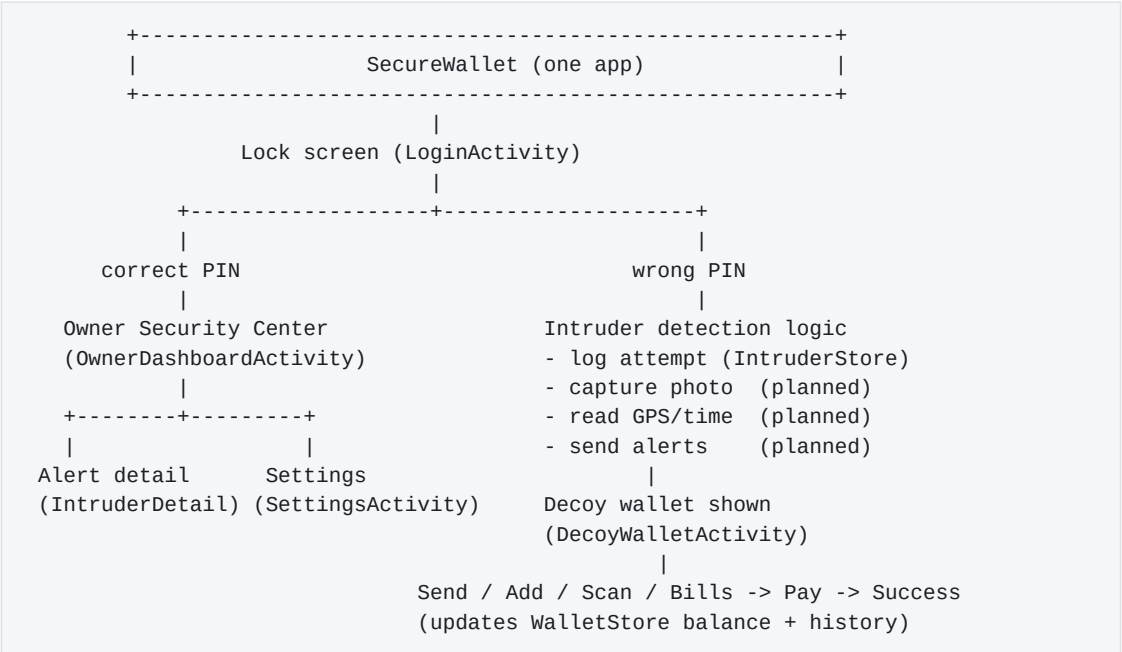

Figure 9.1 - High level flow and architecture of the application.

10. Module and Screen Descriptions

The application is organised into the package `com.example.decoywallet`. Each screen below is a Kotlin Activity backed by an XML layout. The models and the list adapter sit in their own sub packages (model and adapter).

10.1 LoginActivity - the decoy lock screen

This is the launcher screen, so it is what anyone sees first. It draws the SecureWallet branding, the four PIN dots and the keypad. As digits are typed the dots fill in. When four digits are entered the app checks them. If they match the owner PIN it opens the Security Center; otherwise it records an intruder attempt, shakes the dots briefly to look like a normal wrong password response, and then opens the decoy wallet. In the current build a small hint line at the bottom reminds the demonstrator that PIN 4921 is the owner PIN and any other PIN shows the decoy; this line would be removed in a real deployment.

10.2 DecoyWalletActivity - the fake wallet

This is the heart of the deception. It presents a greeting with the account holder name, a balance card showing the current balance in Indian Rupees, a masked card number and the UPI ID, a row of quick actions (Add Money, Send, Scan, Bills) and a Recent Activity list. The balance and the list are read from WalletStore every time the screen is shown, so any fake payment made inside the app is reflected immediately. The four quick actions open their respective screens. The back button is overridden so the intruder cannot navigate back out of the decoy.

10.3 Payment screens - Send, Add Money, Scan, Bills, Pay

These screens make the decoy convincing by actually working (on fake data):

- **SendActivity** - enter a recipient name, a UPI ID or mobile number, an amount and an optional note, then Proceed to Pay. The amount is deducted from the fake balance and a new transaction is added.
- **AddMoneyActivity** - enter an amount (or tap a quick chip like Rs 500, Rs 1000), choose a source such as UPI, Debit Card or Net Banking, and the fake balance goes up.
- **ScanActivity** - shows an animated QR scanner frame, then simulates detecting a merchant and hands over to the payment screen.
- **BillsActivity** - a list of Indian billers and recharges (Mobile, DTH, Broadband, Electricity, Gas, Water, Credit Card), each opening the payment screen with a sensible pre filled amount.
- **PayActivity** - a shared confirm and pay screen used by Scan and Bills, showing the payee and an amount field.

Every one of these ends at **PaymentSuccessActivity**, a shared success screen with a green tick, the amount, the payee, a generated transaction ID, the date and time, and a Done button. This reuse keeps the behaviour consistent and the code small.

10.4 OwnerDashboardActivity - the Security Center

The real owner side. It shows a protection status card, two stat tiles (intruder attempts and alerts sent) and a list of all intruder alerts drawn from IntruderStore using a RecyclerView and IntruderAdapter. Each row shows the attempt, its time and its location. A settings icon in the header opens the Settings screen. Tapping a row opens the full alert details.

10.5 IntruderDetailActivity - one alert in full

Shows everything captured for a single intruder attempt: the snapshot area (a placeholder in this build, where the captured front camera photo would appear), the location with latitude and longitude, the timestamp, the device model and Android version, the number of failed attempts, and the alert channels that were used. A View on Map button is provided for opening the location (planned).

10.6 SettingsActivity - controlling the protection

Lets the owner switch individual protections on or off: capture intruder photo, track location, OTP authentication for owner access, and stealth mode (hiding the app icon from the launcher). It also lets the owner pick which decoy interface to present (banking wallet, photo gallery vault, or notes and passwords) and which alert channels to use (email, SMS, WhatsApp). In this build these controls are UI only; they show how the real settings would be presented and stored.

11. Data Model and State Management

Rather than a full database, the current build keeps its data in two simple Kotlin objects held in memory for the session. This is enough to make every screen behave correctly and makes the logic easy to read. In a production build these would be backed by a local database (such as Room) and a cloud store.

WalletStore

Holds the decoy wallet state: the current balance, the account holder name, the UPI ID and the masked card number, and the list of transactions. It exposes operations like send, addMoney, payMerchant and payBill. Each of these adjusts the balance and inserts a new Transaction at the top of the history. It also contains the Indian Rupee formatting helper, which groups digits the Indian way (for example 1,24,805.56) and adds the rupee symbol.

```
data class Transaction(
    val id: String,
    val name: String,
    val sub: String,
    val amount: Double        // negative = debit, positive = credit
)

object WalletStore {
    var balance: Double = 124805.56
    val transactions: MutableList<Transaction> = mutableListOf( ... )

    fun send(name, upiOrPhone, amount): Transaction      // debit
    fun addMoney(amount, source): Transaction            // credit
    fun payMerchant(name, sub, amount): Transaction      // debit
    fun payBill(biller, amount): Transaction             // debit
    fun formatInr(amount, paise = true): String          // "Rs 1,24,805.56"
}
```

Figure 11.1 - The wallet data model and store (simplified).

IntruderStore and the Intruder model

Holds the list of intruder alerts. Each Intruder record carries an id, a title, a timestamp, a location with coordinates, the device information, the number of failed attempts and the alert channels used. The store seeds a few sample alerts so the Security Center has something to display, and its logAttempt function adds a fresh alert to the top whenever a wrong PIN is entered, which is exactly what the owner would see after a real break in attempt.

12. User Interface Design

The interface had to do two jobs at once: look like a real, trustworthy banking app, and stay comfortable to use. A dark theme was chosen because it reads as modern and premium, which fits a fintech style product, and because it makes the blue accent and the coloured amounts stand out.

The visual language is consistent across every screen: a near black background, slightly lighter cards with soft rounded corners and thin borders, a blue to indigo gradient on the main balance card and primary buttons, and a small set of status colours (green for credit and success, red for alerts and debits, amber for warnings). Icons are simple line icons. Indian context runs through the content: rupee amounts with Indian digit grouping, UPI IDs, and familiar names like Zomato, Jio, Infosys and BESCOM in the transaction list.

One practical design decision is worth recording because it came out of a real problem during development. The login screen was first laid out with ConstraintLayout and GridLayout. During the build, the resource compiler refused to link the constraint and grid attributes in that particular file, even though the libraries were present. Rather than lose time fighting the tooling, the login screen was rebuilt using a plain vertical LinearLayout with the keypad arranged as four rows of three keys. It compiled cleanly, looks identical to the user, and is honestly simpler to read. The other screens use LinearLayout and ScrollView throughout for the same reason: predictability.

13. Security Design and Considerations

Security is the whole point of the project, so the design keeps a few principles in mind even where the implementation is still at the placeholder stage.

Deception as a layer

The first line of defence is that the app does not look like a security tool. Stealth mode is designed to hide the launcher icon so the app is hard to find in the first place, and the decoy wallet means that even a successful guess of the PIN does not reveal anything real.

Silent evidence capture

On a wrong PIN the front camera is meant to fire with no shutter sound, no flash and no preview, so the intruder has no idea their photo was taken. The same trigger collects the location, the time and the device details. The guiding rule is that nothing on screen should change in a way that signals detection.

OTP authentication for owner access

To stop someone who somehow learns the owner PIN, the design includes an optional OTP step for reaching the Security Center, so owner access needs something the owner has (their registered number) in addition to the PIN.

Encryption and secure transmission

The captured evidence is intended to be encrypted before it leaves the device and sent over secure channels (HTTPS/TLS), with an optional encrypted copy stored in the cloud. This protects the evidence both while it is being sent and while it is stored.

What is implemented now

In this build the detection decision, the attempt logging, the decoy and the owner review flow are all real and working. The camera capture, the location read, the alert sending and the encryption are represented by clearly marked placeholders in the code (search for the comment TODO (full app)), so it is obvious where each real service attaches. This separation was intentional, so the app can be run and shown safely without requesting camera, location or network permissions during a demo.

14. Development and Build Process

The project is a standard Android Studio / Gradle project. It can be opened directly in Android Studio, which handles the Gradle sync and downloads the dependencies automatically. To build and run it from the command line, the steps are:

```
# 1. Point Gradle at the Android SDK (once)
echo "sdk.dir=$HOME/Library/Android/sdk" > local.properties

# 2. Build the debug APK
./gradlew assembleDebug
#    -> app/build/outputs/apk/debug/app-debug.apk

# 3. Install on a running emulator or a connected phone
adb install -r app/build/outputs/apk/debug/app-debug.apk

# 4. Launch it
adb shell monkey -p com.example.decoywallet -c android.intent.category.LAUNCHER 1
```

Figure 14.1 - Building and running the app.

During development the app was built with Gradle 8.7 and run on an Android emulator (a Pixel 6a running API 35). The whole decoy flow was tested there: entering a wrong PIN to reach the decoy wallet, sending a fake payment and watching the balance and history update, and entering the owner PIN to reach the Security Center and open an alert.

15. Testing

Testing was done manually on the emulator, screen by screen and flow by flow. The main cases are listed below.

#	Test case	Expected result	Status
1	Enter owner PIN (4921)	Opens the Security Center	Pass

#	Test case	Expected result	Status
2	Enter any wrong PIN	Logs an attempt, shows the decoy wallet	Pass
3	Send money in decoy wallet	Balance reduces, new debit appears on top	Pass
4	Add money in decoy wallet	Balance increases, new credit appears	Pass
5	Scan and pay	Scanner animates, merchant detected, payment succeeds	Pass
6	Pay a bill / recharge	Opens pay screen pre-filled, payment succeeds	Pass
7	Open an intruder alert	Shows snapshot area, location, time, device	Pass
8	Back button in decoy wallet	App goes to background, does not return to login	Pass
9	Currency formatting	Amounts show Indian grouping, e.g. 1,24,805.56	Pass
10	Rupee balance after a payment	Reflects the exact debited/credited amount	Pass

Table 15.1 - Manual test cases and results.

16. Project Folder Structure

```
DecoyWallet/
  app/
    src/main/
      AndroidManifest.xml
      java/com/example/decoywallet/
        LoginActivity.kt            # decoy lock screen (launcher)
        DecoyWalletActivity.kt      # fake wallet shown to intruder
        SendActivity.kt             # send money
        AddMoneyActivity.kt         # add money
        ScanActivity.kt             # QR scan (simulated)
        BillsActivity.kt            # bills & recharge list
        PayActivity.kt              # shared confirm-and-pay
        PaymentSuccessActivity.kt   # shared success screen
        OwnerDashboardActivity.kt   # owner Security Center
        IntruderDetailActivity.kt   # one alert in detail
        SettingsActivity.kt         # protection settings
        adapter/IntruderAdapter.kt  # list adapter for alerts
        model/Intruder.kt           # Intruder model + IntruderStore
        model/Wallet.kt             # Transaction model + WalletStore
      res/
        layout/     # one XML per screen + reusable rows
        drawable/   # icons, gradients, card backgrounds, shapes
        values/     # colors, strings, styles, themes
  build.gradle.kts, settings.gradle.kts, gradle/ ...
```

Figure 16.1 - The project layout.

17. Limitations

Being honest about the current limitations:

- The camera capture, GPS, alert sending and cloud upload are designed and marked in the code but not yet wired to the real Android APIs and services.
- The wallet balance and transaction history, and the intruder alerts, live in memory for the session, so they reset when the app process is killed. Persistence (a local database and cloud sync) is future work.
- Stealth mode (hiding the launcher icon) and OTP for owner access are shown as settings but not enforced in this build.
- On very old Android versions some newer layout attributes are ignored, though the app still runs because the minimum is Android 7.0.

18. Future Enhancements

1. Wire up the real silent front camera capture using CameraX, and the real location read using the Fused Location Provider.
2. Integrate real alert delivery over email, SMS and WhatsApp, and an encrypted upload to a cloud backend such as Firebase.
3. Add local persistence with Room so the wallet state and the alert history survive restarts, and sync alerts to the owner's account.
4. Enforce stealth mode and OTP based owner login.
5. Offer more than one decoy interface (gallery vault, notes vault) as hinted in Settings.
6. Add optional extreme-case actions such as a loud alarm, remote lock, or a remote wipe of sensitive data, controlled by the owner.
7. Capture additional evidence such as a short audio clip or repeated location pings while the intruder is active.

19. Ethical and Legal Considerations

An app that photographs a person and records a location without their knowledge has to be handled responsibly. The intended and only use of this system is defensive: it runs on the owner's own device and only reacts to someone attempting unauthorised access. The evidence it gathers is sent to that same owner to help recover the device or support a police complaint.

Anyone deploying such a system should keep the data minimal and secure, use it only for protecting their own property, and respect local laws on recording and privacy. It should never be used to spy on another person or installed on a device that is not the user's own. The design choices in this project (owner only access to the evidence, encryption of that evidence, and capture triggered strictly by a failed unauthorised unlock) are all meant to keep the tool on the right side of that line.

20. Conclusion

The Decoy Wallet Application turns the usual idea of phone security around. Instead of simply blocking an intruder, it hides in plain sight as an ordinary UPI wallet, quietly gathers evidence the instant someone tries a wrong PIN, warns the real owner, and then keeps the intruder busy with a believable fake wallet. The build described here delivers the full application experience, both faces of the app, the working decoy wallet with its payment flows, and the owner's Security Center, with the sensitive hardware and network pieces cleanly designed and marked for integration.

The result is a practical, easy to demonstrate example of proactive, deception based mobile security that is genuinely useful for anyone who wants a stronger answer to phone theft than a lock screen alone. It is also a solid base to build on, with a clear path from the current front end to a fully connected system.

21. References

1 Android Developers - Official documentation. https://developer.android.com
2 Kotlin Programming Language - Official documentation. https://kotlinlang.org/docs
3 Material Design 3 Guidelines. https://m3.material.io
4 Android Developers - CameraX overview. https://developer.android.com/training/camerax
5 Android Developers - Fused Location Provider API. https://developer.android.com/training/location
6 Firebase Documentation (Cloud storage and authentication). https://firebase.google.com/docs
7 Project source: Android application package com.example.decoywallet (this project).